CERTIFICATE

ACKNOWLEDGEMENT

The completion of internship and project involves the efforts of many people. I have been lucky to have received a lot of help and support from all directions during the making of this project, so with gratitude we take this opportunity to acknowledge all those whose guidance and encouragement helped us to emerge successful.

I wish to express thanks to our beloved Principal, **Dr. Girish D P B.E, M.Tech, Ph.D.,** for encouragement throughout my studies and technically sound in our institution.

I express my gratitude to **Dr. Vani V G B.E, M.Tech, Ph.D.**, Associative Professor and Head of the Department of CS & E, for their encouragement and support throughout the work.

At the outset I express my most sincere grateful thanks to my Guide and Coordinator, **Dr. Vasantha Kumara M B.E, M.Tech, Ph.D.** Associate Professor, Department of CS&E, for their continuous support and advice not only during the course of the project but also during the period of my stay in GECH as Internship Guide and Coordinator.

Finally, I express my gratitude to all teaching and non-teaching staff of Department of CS&E, my fellow classmates and my parents for their timely support and suggestions.

ABSTRACT

This report documents the successful completion of a Java Full Stack Web Development internship at ParvaM Software Solutions during the academic year 2024–2025. The primary objective of the internship was to gain hands-on experience in building robust, scalable, and user-friendly web applications using modern full-stack technologies. As part of the internship, a real-time project titled **"Dental Tool Management System"** was developed to address the challenges faced by dental clinics and suppliers in efficiently managing dental equipment inventory, procurement, and distribution.

The system was implemented using Java, Spring Boot, and MySQL for the backend, and HTML, CSS, JavaScript, Thymeleaf, and Bootstrap for the frontend. The application provides essential features like role-based user authentication, comprehensive product catalog management, shopping cart functionality, order processing and tracking, inventory management with low stock alerts, customer reviews and ratings, and advanced search and filtering capabilities. The architecture follows a modular and maintainable design, utilizing concepts such as RESTful APIs, MVC framework, object-relational mapping with Hibernate, and layered service logic with clear separation of concerns.

Through this internship, the student developed proficiency in core Java, object-oriented programming, web technologies, and backend frameworks, gaining insights into the full software development life cycle from requirement analysis to deployment. The experience not only enhanced technical competencies but also fostered problem-solving, teamwork, and project management skills, laying a strong foundation for a future career in full-stack web development. The Dental Tool Management System project provided valuable domain knowledge in healthcare systems and e-commerce applications, demonstrating the practical application of software engineering principles to solve real-world business challenges.

ABOUT THE ORGANISATION

ParvaM Software Solutions is a technology-driven organization that offers innovative business development ideas and effective software solutions using cutting-edge technologies. The company focuses on delivering unique experiences in technology, consulting, and software services by implementing enterprise applications, professional websites, mobile applications, and customized web services. ParvaM also conducts technology workshops on various trending technologies to empower businesses with confidence. Known for its in-time deliverables, the organization follows a strategic approach that includes conceptual planning, flexible technology choices, wireframe designing, optimized development, and post-launch support, ensuring high-quality solutions and customer satisfaction.

HISTORY

ParvaM Software Solutions has established itself by serving customers with a delighting experience through its expert services and commitment to excellence. With a competitive edge in delivering unique experiences in technology, consulting, and software solutions, ParvaM has been actively involved in implementing enterprise applications, professional websites, mobile applications, and customized web services. The organization has demonstrated a consistent track record of timely deliverables and strategic advancements, leveraging cutting-edge technologies and innovative business ideas. Through technology workshops and flexible solution planning, ParvaM has contributed to the modernization and growth of its clients' businesses, reflecting its evolution into a trusted and forward-thinking software solutions provider.

Vision: To be a leading provider of innovative and reliable technology solutions that drive business growth and transformation.

Mission: To deliver exceptional software services and consulting through cutting-edge technologies, timely execution, and customer-centric approaches.

TABLE OF CONTENTS

LIST OF FIGURES

CHAPTER 1

INTRODUCTION

1.1 ABOUT THE INTERNSHIP

The internship at **ParvaM Software Solutions** was undertaken to gain practical experience in full stack web development using Java technologies. The program aimed to bridge the gap between academic knowledge and industry requirements by offering real-time exposure to web application development, covering both frontend and backend technologies. Over the course of the internship, interns were trained in core Java, web technologies like HTML, CSS, and JavaScript, as well as advanced frameworks including Spring Boot and Hibernate.

1.2 OBJECTIVE OF THE INTERNSHIP

The primary goal of the internship was to build a strong foundation in full stack development, understand enterprise application architecture, and apply the concepts learned to a real-world project. The project assigned during the internship—**Indoor Sports Slot Booking System**—was a complete web application that enabled users to book slots for various indoor games and allowed administrators to manage bookings, users, and sports activities.

1.3 LEARNING OUTCOMES

Through this internship, the following outcomes were achieved:

- Gained hands-on experience in full stack development.
- Understood MVC architecture and how to build RESTful services.
- Implemented CRUD operations using Spring Boot and JPA.
- Learned how to integrate frontend and backend components effectively.
- Developed a functional, responsive, and user-friendly web application.

CHAPTER 2

CORE JAVA

2.1 INTRODUCTION TO CORE JAVA

Core Java is the fundamental part of the Java programming language, focusing on the basic concepts and features that form the foundation for all Java-based development. It includes essential topics like object-oriented programming (OOP), data types, control structures, arrays, classes and objects, inheritance, polymorphism, interfaces, exception handling, multithreading, and collections. Core Java is platform-independent, meaning code written in Java can run on any system with a Java Virtual Machine (JVM). It is widely used for building desktop applications, backend systems, and as a stepping stone to more advanced Java technologies like Java EE, Android development, and Spring Framework.

2.2 GENERATION

In the context of programming languages like Java, generation typically refers to the different versions or evolutions of the language, its runtime environment, and its associated technologies. Java has evolved through multiple generations, each bringing new features,

improvements, and enhancements to the language and its ecosystem. Below is a brief overview of the major generations or versions of Java:

1. Java 1.0 (1996) – The First Generation

- This was the initial release of Java, which introduced fundamental concepts like object- oriented programming, platform independence (write once, run anywhere), and the Java Virtual Machine (JVM).
- Java 1.0 was mainly designed for applets (small applications embedded in web browsers), but it lacked many features like GUI support and extensive libraries.

2. Java 2 (J2SE) (1998) – The Second Generation

- Java 2 was a significant upgrade, which included the introduction of Swing for building graphical user interfaces (GUIs) and the Java 2 Platform, Enterprise Edition (J2EE) for enterprise applications.

- It also introduced the Java 2 Platform, Micro Edition (J2ME), designed for mobile and embedded devices.
- This release marked the real emergence of Java as a widely adopted programming language.

3. Java 5 (J2SE 5.0) (2004) – The Third Generation

- A major milestone, Java 5 introduced language features like generics, metadata annotations, enumerated types, varargs, and the enhanced for loop.
- This release dramatically improved the language's power and flexibility, making it more efficient for developers working on complex systems.

4. Java 6 (2006) – The Fourth Generation

- Focused on performance improvements, including enhancements to the Java Virtual Machine (JVM) and updates to the Java compiler.
- Java 6 also introduced scripting support via the Java Compiler API and Java Compiler Service (JSR 199) as well as improvements in the Java DB (Derby).

5. Java 7 (2011) – The Fifth Generation

- Java 7 included new language features such as the try-with-resources statement for better resource management, diamond operator for simplifying generics, and support for NIO 2.0 (New Input/Output API).

6. Java 8 (2014) – The Sixth Generation

- One of the most significant versions, Java 8 introduced Lambda expressions, which enabled functional programming features in Java.
- It also introduced the Stream API for processing sequences of elements and new Date and Time API (java.time), which offered better handling of date and time.
- These features made Java more modern and suitable for parallel programming and functional paradigms.

7. Java 9 (2017) – The Seventh Generation

- Java 9 brought the long-awaited modular system (Project Jigsaw) to Java, allowing developers to create modular applications.
- It also introduced the JShell tool for interactive Java programming and improvements to the Java compiler and JVM.

7. Java 10 (2018) and Java 11 (2018) – The Eighth and Ninth Generations

- Java 10 introduced local-variable type inference (the var keyword) to simplify coding and reduce verbosity.
- Java 11 focused on long-term support (LTS), with new features like HTTP Client API, enhanced garbage collection, and deprecated Java EE and CORBA modules.
- Java 11 is also an LTS release, making it a preferred choice for enterprises looking for stability.

8. Java 12 and Beyond (2019-2025) – The Latest Generations

- Since Java 9, new versions have been released more frequently (every six months), bringing incremental improvements.
- Java 12 introduced features like JVM Constants API and Shenandoah garbage collector.
- Java 14 introduced record types (a new way to model data) and pattern matching.
- As of Java 17 (LTS), Java has focused on enhancing performance, security, and usability, as well as making functional programming concepts more integral to the language.

2.3 OBJECT ORIENTED PROGRAMMING

Object-Oriented Programming (OOP) is a programming paradigm based on the concept of objects, which are instances of classes. It aims to model real-world entities and their interactions through a set of principles that make code more modular, reusable, and easier to maintain. Here are the key concepts in OOP:

1. Classes and Objects:

A class is a blueprint or template for creating objects. An object is an instance of a class and has both data (attributes) and methods (functions or behaviors) that operate on

on that data.

2. Encapsulation:

Encapsulation is the concept of bundling the data (attributes) and the methods (functions) that operate on the data into a single unit or class. It also involves restricting direct access to some of an object's components, which helps protect its integrity by controlling how data is accessed and modified.

3. Inheritance:

Inheritance is a mechanism that allows one class to inherit properties and behaviors from another. The new class, called the subclass or derived class, can use or override the functionality of the existing class, called the superclass or base class. This promotes code reusability.

4. Polymorphism:

Polymorphism allows one entity to take multiple forms. In OOP, polymorphism typically refers to the ability to redefine methods in subclasses (method overriding) or to use a single method to operate on different types of objects (method overloading). This enables flexibility and dynamic behavior.

5. Abstraction:

Abstraction involves hiding the complex implementation details and showing only the essential features of an object. It allows the programmer to focus on what an object does rather than how it does it. This is typically achieved through abstract classes or interfaces.

2.4 VARIABLES AND DATA TYPES

Variables

A variable in Java is used to store data that can be referenced and manipulated in a program. Each variable is associated with a data type which defines the kind of data it can hold.

Types of Variables:

1. Instance Variables: Declared inside a class but outside any method. These variables are tied to a specific instance of a class (object).
2. Class Variables (Static Variables): Declared with the static keyword, these variables belong to the class rather than to any particular object.
3. Local Variables: Declared inside a method, constructor, or block and can only be used within that scope.
4. Parameter Variables: Declared in method definitions to hold arguments passed to the method.

Data Types

1. Primitive Data Types

These are the basic data types in Java that hold simple values. They are not objects and store values directly.

- byte: 8-bit signed integer, range: -128 to 127.
- short: 16-bit signed integer, range: -32,768 to 32,767.
- int: 32-bit signed integer, range: -2^31 to 2^31-1 (typically used for integers).
- long: 64-bit signed integer, range: -2^63 to 2^63-1 (used for large integers).
- float: 32-bit floating-point number (used for decimal values, with single precision).
- double: 64-bit floating-point number (used for decimal values, with double precision).
- char: 16-bit Unicode character (used for storing single characters).
- boolean: Holds true or false values.

2. Reference (Non-Primitive) Data Types

These are more complex types that refer to objects and store references to memory locations.

- String: A sequence of characters (not a primitive type but widely used).
- Arrays: A collection of elements of the same type.

- Classes and Objects: Custom data types created by the user.

2.5 CONTROL FLOW

Control flow in Java determines the order in which individual statements, instructions, or function calls are executed. It allows the program to make decisions, repeat actions, and alter its normal sequence of execution based on conditions. The key components of control flow in Java are:

1. Conditional Statements: These allow the program to execute different blocks of code based on certain conditions.

- The if statement evaluates a condition and executes a block of code if the condition is true.
- The if-else statement provides an alternative block of code to execute if the condition is false.
- The switch statement allows a variable to be compared against multiple values, making it ideal for handling many conditions based on a single variable.

2. Looping Statements: These enable the program to repeat a block of code multiple times based on a condition.

- The for loop is typically used when the number of iterations is known.
- The while loop repeats a block of code as long as a specified condition is true.
- The do-while loop is similar to the while loop, but it guarantees at least one execution of the loop.

3. Branching Statements: These are used to alter the flow of control in loops or switch statements.

- The break statement immediately exits the current loop or switch statement.
- The continue statement skips the current iteration of a loop and moves to the next one.

4. Ternary Operator: A shorthand form of the if-else statement, the ternary operator evaluates a condition and returns one value if the condition is true and another value if false.

2.6 METHODS

Method Declaration: A method is defined by its name, return type, and parameters (optional). The return type specifies the type of value the method will return (e.g., int,

1. String), or void if it does not return a value. The method name is used to invoke it, and parameters are the inputs that the method can accept to perform its task.

2. Method Call: To execute a method, you simply call it from another method or class by using its name and passing any required parameters (if applicable). A method can be invoked multiple times, making it reusable.

3. Return Statement: Methods can return values using the return keyword. If the method has a return type (e.g., int, String), it must return a value of that type. If the method is declared with void, it does not return any value.

4. Method Overloading: Java allows you to define multiple methods with the same name but with different parameters. This is called method overloading and enables you to perform similar actions with different input types or numbers of parameters.

5. Method Overriding: Method overriding occurs when a subclass provides its own implementation of a method that is already defined in its superclass. This allows the subclass to modify or extend the behavior of the inherited method.

6. Static Methods: A static method is associated with the class itself rather than any specific instance (object) of the class. Static methods are often used for operations that don't depend on object state and can be called using the class name.

7. Void Methods: A void method is a method that does not return any value. These methods are often used for performing actions or tasks that don't need to return data, such as printing a message or modifying the state of an object.

2.7 EXCEPTION HANDLING

Exception handling in Java is a mechanism to manage runtime errors and maintain the program's normal flow. Key concepts include:

1. Exceptions: Events that disrupt normal execution, which can be checked (compile-time) or unchecked (runtime).
2. Try-Catch Block:
3. try: Contains code that may throw exceptions.
4. catch: Catches and handles the exception if thrown.
5. Finally Block: Executes code regardless of whether an exception occurs, often used for cleanup.
6. Throw Statement: Explicitly throws an exception.
7. Throws Keyword: Declares that a method may throw exceptions, and they must be handled by the caller.
8. Checked vs. Unchecked:
9. Checked exceptions (e.g., IOException) are checked at compile-time.
10. Unchecked exceptions (e.g., NullPointerException) are checked at runtime.

2.8 MULTITHREADING

Multithreading in Java is a programming feature that allows the simultaneous execution of two or more parts of a program for maximum utilization of CPU. A thread is a lightweight sub-process and is the smallest unit of processing. Java supports multithreading through the

Thread class and the Runnable interface. By using multithreading, a program can perform multiple operations at the same time, making it more efficient and responsive, especially in tasks involving I/O, user interfaces, or background processing.

Each thread in Java goes through a life cycle that includes several states: New, Runnable, Running, Blocked or Waiting, and Terminated. The main methods associated with threads are start() to begin execution, run() to define the task, sleep() to pause a thread, join() to wait for another thread to finish, and isAlive() to check if thread is still running. Since threads often share resources, Java provides synchronization mechanisms to prevent issues like race conditions. The synchronized keyword is used to ensure that only one thread can access a critical section at a time.

In Java, interfaces and abstract classes are used to achieve abstraction, a core principle of object-oriented programming that hides implementation details and shows only functionality. An interface in Java is a reference type that can contain only constants,

method signatures, default methods, static methods, and nested types. All methods in an interface are implicitly public and abstract (except static and default methods). Interfaces are used when multiple classes need to share a common set of behaviors but are otherwise unrelated. Java supports multiple inheritance through interfaces, allowing a class to implement multiple interfaces.

An abstract class, on the other hand, is a class that cannot be instantiated and is declared using the abstract keyword.

CHAPTER 3

HTML

3.1 BASIC STRUCTURE OF AN HTML DOCUMENT

The basic structure of an HTML document is essential for creating a webpage. It follows a standardized format that includes various elements, each serving a unique purpose in organizing the content. Here's a breakdown of the components:

1. <!DOCTYPE html>

- The DOCTYPE declaration is the first thing in an HTML document. It tells the web browser what version of HTML the page is written in.

2. <html>

- The <html> tag is the root element of an HTML document. It contains all the content on the page. It can also include an attribute called lang, which specifies the language of the document (e.g., <html lang="en"> for English).

3. <head>

- The <head> element contains metadata about the HTML document. This metadata is not visible on the webpage but plays an important role in how the page is interpreted by the browser and search engines. Common tags inside the <head> include:
- <meta>: Specifies the character encoding (e.g., <meta charset="UTF-8">) and viewport settings for responsive design.
- <title>: Sets the title of the webpage, which appears in the browser's title bar or tab.
- <link>: Links external resources like CSS stylesheets.
- <script>: Includes JavaScript files or inline scripts.

4. <body>

- The <body> tag contains the content that is visible on the webpage. Everything inside the body tag is what the user will see when they visit the page.

Common HTML tags

- Headings: <h1> to <h6> – Define titles and subheadings.
- Paragraphs: – Define text blocks.
- Links: <a> – Create hyperlinks.
- Images: <img> – Embed images.
- Lists: <ul>, <ol>, <li> – Create bullet or numbered lists.
- Tables: <table>, <tr>, <td>, <th> – Organize data in rows and columns.
- Forms: <form>, <input>, <textarea>, <button> – For user input.
- Divisions and Spans: , – Used for layout and styling.

3.2 FEATURES OF HTML

Easy to Learn: Simple syntax and structure.

Platform Independent: Works on all browsers and devices.

Extensible: Can be enhanced using CSS for styling and JavaScript for behavior.

Search Engine Friendly: Proper HTML helps search engines read and index

pages. SUMMARY

HTML (HyperText Markup Language) is the fundamental language used to create and structure content on the web. It defines the structure of web pages using a system of tags, which tell the web browser how to display various elements such as text, images, videos, links, forms, and more. HTML documents are composed of key components like the

<!DOCTYPE html> declaration, the <html> root element, the <head> for metadata, and the

<body> for the visible content. HTML tags are generally written in pairs, such as <h1> and

</h1>, to define headings, or and for paragraphs. HTML also supports attributes like href for links or src for images, which provide additional information about the elements. As the foundation of web development, HTML works alongside CSS for styling and JavaScript for interactivity, making it essential for creating functional, well-designed websites. Its simplicity, platform independence, and widespread support across web browsers make HTML a crucial skill for anyone involved in web development.

CHAPTER 4

CSS

4.1 INTRODUCTION

CSS (Cascading Style Sheets) is a stylesheet language used to describe the presentation of a document written in HTML or XML. While HTML provides the structure of a webpage, CSS is responsible for its visual layout and design. CSS allows web developers to separate the content from its design, making websites easier to manage, style, and maintain.

CSS controls various visual elements such as colors, fonts, spacing, alignment, and layout, providing flexibility and creativity in how web pages look. By using CSS, developers can ensure that their websites have consistent styling across different pages, creating a more polished and user-friendly experience.

CSS is essential for creating responsive and adaptable websites that work across different devices and screen sizes, ensuring that the content looks good on everything from desktop computers to mobile phones. Whether for changing font styles, adding background images, or defining the layout, CSS plays a crucial role in modern web development.

Through CSS, developers can design engaging, visually appealing websites while keeping the structure (HTML) separate from the style and layout. This separation simplifies web development, allows for faster updates, and provides better organization for large websites.

4.2 BASIC STRUCTURE OF CSS

CSS (Cascading Style Sheets) is used to control the appearance and layout of web pages. It allows developers to define how HTML elements should be displayed in terms of color, font, spacing, positioning, and other visual aspects. The basic structure of CSS consists of rules that define styles for various elements of a webpage.

A CSS rule is made up of three primary parts:

1. Selector: The selector is used to target an HTML element that you want to style. It can be an HTML tag, class, id, or attribute. The selector specifies which HTML elements the style rule should apply to.Property: The property defines what aspect of the HTML element you want to style. Properties can control various features such as text color (color), font size (font-size), background color (background-color), borders, margins, padding, and many more.

2. Value: The value specifies the specific setting for the property. For instance, for the property color, the value could be a color name like red or a hexadecimal value like #ff0000. For properties like font-size, the value could be a size like 12px or 1em.

CHAPTER 5

JAVA SCRIPT

5.1 FEATURES

- Client-Side Scripting:

JavaScript is mainly used for client-side development, meaning it runs in the user's browser, allowing for real-time interaction without needing to reload the page. This makes websites more dynamic.

- Dynamic Typing:

JavaScript is a loosely typed language, meaning you don't have to declare the type of variables (such as integers or strings) when they are created. This provides flexibility in writing code.

- Event-Driven:

JavaScript is event-driven, meaning it can respond to various user actions, such as clicks, mouse movements, or keystrokes, without needing to refresh the page.

- Asynchronous Programming:

JavaScript supports asynchronous programming, allowing for non-blocking operations, such as loading data from a server (using AJAX), enabling a smoother user experience.

- Cross-Platform Compatibility:

JavaScript is supported by all modern browsers (like Chrome, Firefox, Safari, etc.) and works across different operating systems, making it an essential tool for web development.

- Versatility:

JavaScript can be used for a variety of purposes, including form validation, animations, interactive maps, and even server-side development using Node.js.

5.2 BASIC STRUCTURE OF JAVASCRIPT

1. Comments

- Comments are used to make the code more understandable to humans. They are ignored by the JavaScript engine and are commonly used to explain the purpose of specific code sections.

2. Variables

- Variables are containers for storing data values. JavaScript provides different keywords to declare variables, each with its own scoping rules. These variables can hold various data types and can be updated or reassigned, depending on how they are declared.

3. Data Types

- JavaScript supports multiple data types including:
- Primitive types such as strings, numbers, booleans, null, and undefined.
- Complex types such as objects and arrays. Understanding the nature of each type is crucial for managing data and memory effectively.

4. Operators

- Operators are symbols used to perform operations on variables and values. They include arithmetic operators, comparison operators, logical operators, and assignment operators, which are essential for expressing logic and computation in programs.

5. Control Structures

- Control structures determine the flow of execution in a program. Conditional statements (like if-else) allow branching based on certain conditions. Looping structures (such as for and while) are used to execute blocks of code multiple times, often based on a defined condition.

6. Functions

- Functions are reusable blocks of code that perform a specific task. They help in organizing code, avoiding repetition, and enhancing maintainability.

CHAPTER 6

FRAMEWORKS

6.1 INTRODUCTION

Frameworks are essential tools in modern web development. They provide developers with a structured foundation for building interactive, scalable, and maintainable web applications. Instead of writing all functionality from scratch, frameworks offer pre-built components, tools, and best practices to streamline the development process.

At their core, JavaScript frameworks simplify tasks such as user interface rendering, data binding, event handling, form validation, routing, and communication with web servers. They promote code reusability, consistency, and efficiency, especially when working on large or complex applications.

Frameworks vary in scope and purpose—some focus on the front end (like React, Angular, Vue), while others serve the backend (like Node.js with Express). Front-end frameworks handle what the user sees and interacts with, while back-end frameworks manage server-side operations, databases, and APIs.

As web applications have grown in complexity and user expectations have risen, the use of frameworks has become a standard practice in the software development industry. By providing a clear structure and robust functionality, JavaScript frameworks help developers build better applications in less time, with fewer bugs and easier maintenance.

6.2 HIBERNATE

Hibernate is a popular open-source **Object-Relational Mapping (ORM) framework** for Java. It simplifies the development of Java applications that interact with databases by mapping Java classes to database tables, allowing developers to work with databases using standard Java objects rather than SQL queries.

In traditional JDBC (Java Database Connectivity), developers need to write complex SQL queries and manage connections, statements, and result sets manually. Hibernate abstracts much of this boilerplate code, providing a more efficient and cleaner way to perform

6.3 INTRODUCTION TO SPRING BOOT

Spring Core is the foundation of the Spring Framework, a powerful, flexible, and comprehensive framework for building Java-based applications. It provides comprehensive infrastructure support for developing Java applications, with particular focus on ease of use, flexibility, and testability. The core modules of Spring manage the infrastructure and provide a solid foundation for building Java applications.

Spring Core primarily addresses dependency injection (DI) and aspect-oriented programming (AOP), which are two key concepts that promote loose coupling, maintainability, and flexibility in software development.

6.4 CONCEPTS OF SPRING CORE

1. Inversion of Control (IoC)

In traditional application development, objects are responsible for creating and managing other objects. IoC is a design principle in which the control of object creation and management is inverted, meaning that it's handled by a container rather than the objects themselves. Spring IoC is implemented through Dependency Injection (DI), where the Spring container injects dependencies into objects rather than having objects create them. This reduces tight coupling between components and makes the code easier to test and Maintain.

Dependency Injection (DI):

DI is a core concept of Spring IoC, where the dependencies of an object (e.g., services, data access objects) are injected by the Spring container rather than being hardcoded within the object. DI can be performed in the following ways:

- Constructor Injection: Dependencies are provided through the constructor.
- Setter Injection: Dependencies are provided through setter methods.
- Field Injection: Dependencies are directly injected into fields.

2. Bean Definition and Configuration

In Spring, objects are called beans, and the configuration of these beans

(including how they are created and their dependencies) is managed by the Spring IoC container. Beans are typically defined in XML files or through annotations in the code, allowing developers to declare how objects should be created, their lifecycle, and their dependencies.

3. ApplicationContext

The ApplicationContext is the central interface to the Spring IoC container. It is responsible for managing the beans and handling their lifecycle. It can be thought of as the heart of the Spring container, providing support for resolving bean dependencies, configuration, and management of resources like database connections, services, and more.

CHAPTER 7

PROJECT IMPLEMENTATION

7.1 INTRODUCTION

- A Dental Tool Management System is an integrated software solution designed to manage the inventory, procurement, and distribution of dental equipment and tools efficiently.
- **Objective:** To digitize dental equipment management processes, enhance productivity, reduce paperwork, and enable real-time access to important inventory and order data.

Key Features:

- Product & Category Management
- User Role Management
- Order Processing & Tracking
- Inventory Management
- Review & Rating System
- Shopping Cart Functionality
- Reporting & Analytics

7.2 IMPLEMENTATION

BACKEND Implementation:

1. **Technologies used**: Java, Spring Boot, MySQL
2. **Key Components:**

Entity Classes:

- User: Stores user information with roles (Admin, Staff, Customer)
- Product: Contains details about dental equipment
- Category: Organizes products into logical groups
- Cart & CartItem: Manages shopping cart functionality

- Order & OrderItem: Tracks customer orders
- Address: Stores shipping and billing information
- Contact: Handles customer inquiries
- ProductReview: Manages customer reviews and ratings

REPOSITORIES:

- Using JpaRepository for CRUD operations
- Custom query methods for specific data retrieval needs

These repositories extend `JpaRepository` to support built-in and custom queries using Spring Data JPA.

CONTROLLERS:

- REST APIs for admin operations
- Web controllers for page rendering
- Specialized controllers for product, order, and user management

SERVICES

- Business logic for managing products, orders, and users
- Authentication and authorization services
- Shopping cart and checkout processing
- Review and rating management

DATABASE:

- MySQL with relations between entities
- Proper indexing for performance optimization
- Foreign key constraints for data integrity

FRONTEND IMPLEMENTATION

Features Implemented:

- Dashboard with total products/orders/users
- Forms for adding/viewing/editing records

- Product catalog with filtering and search
- Shopping cart with real-time updates
- Order management section with date filters
- Responsive UI with Bootstrap

Technologies:

- HTML + Thymeleaf + Bootstrap
- JavaScript for dynamic interactions
- CSS for custom styling

1. Security Implementation:

- Spring Security for authentication and authorization
- Role-based access control
- Password encryption
- CSRF protection

2. Shopping Cart Implementation:

- Session-based cart for non-logged in users
- Database-persistent cart for logged in users
- Real-time cart updates and calculations

3. Order Processing:

- Multi-step checkout process
- Order status tracking
- Email notifications for order updates

4. Product Management:

- Category-based organization
- Image upload and management
- Inventory tracking with low stock alerts

5. Review System:

- Star-based rating system
- Text reviews with moderation
- Average rating calculation

Version Control:

Used Git for source code management with feature branching and pull request workflows.

7.3 METHADOLOGY

1. Requirement Analysis and Planning
    - Analyzed existing dental equipment management solutions to identify gaps and opportunities for improvement.
    - Created detailed functional and non-functional requirements documents, including user stories and use cases for different user roles (Admin, Supplier, Customer).
    - Clearly defined project boundaries, deliverables, and success criteria in collaboration with the internship mentor.
2. System Design
    - Created a three-tier architecture (presentation, business, and data layers) with clear separation of concerns.
    - Defined RESTful API endpoints and specifications for communication between frontend and backend components.
    - Established security protocols including authentication, authorization, data encryption, and secure communication channels.
3. Implementation
    - Implemented fronten using the technologies like html, css and javascript.
    - Created business logic for inventory management, order processing, and user authentication.
4. Testing
    - Performed integration tests to verify the interaction between different system components.
    - Evaluated system performance under various load conditions
5. Deployment
    - Created detailed deployment documentation and checklists.
    - Configured production environment settings, including security parameters and performance optimizations.

CHAPTER 8

RESULTS

8.1 HOME PAGE

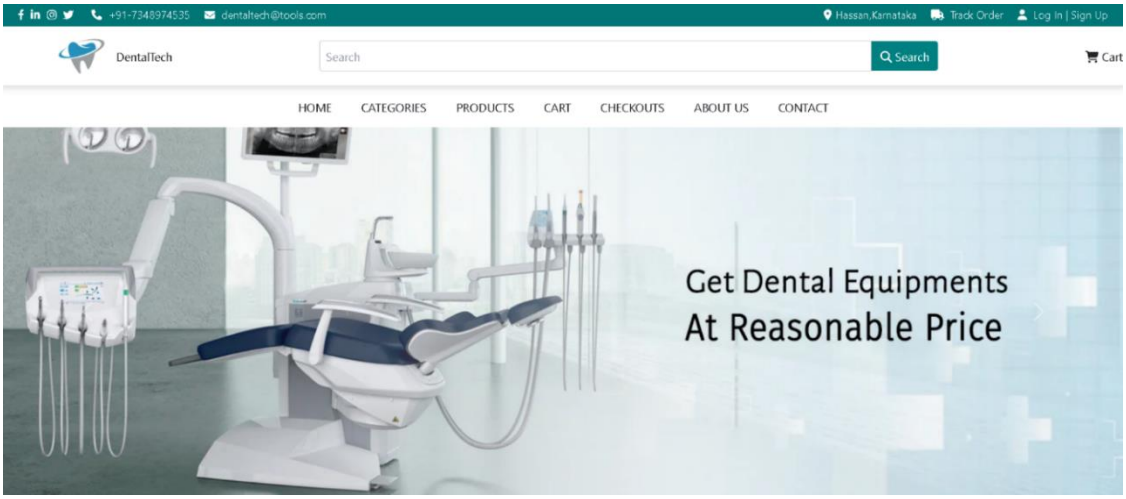

Figure 8.1: *Home Page*

8.2 LOGIN PAGE

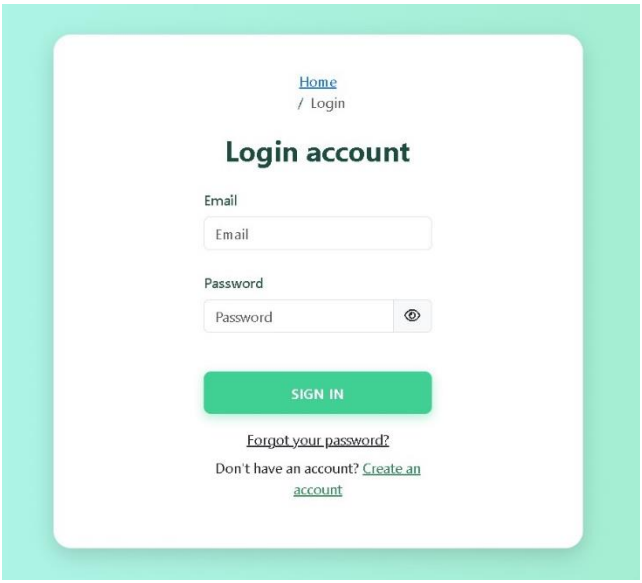

Figure 8.2: *Login Page*

8.3 SIGNUP PAGE

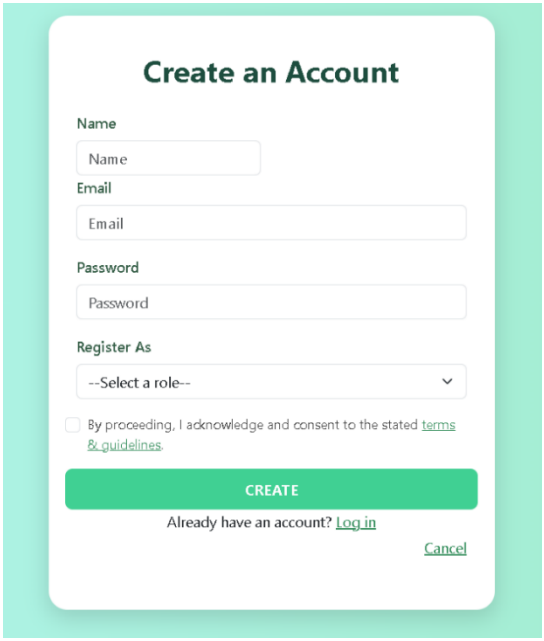

Figure 8.3: *Signup Page*

8.4 CATEGORY PAGE

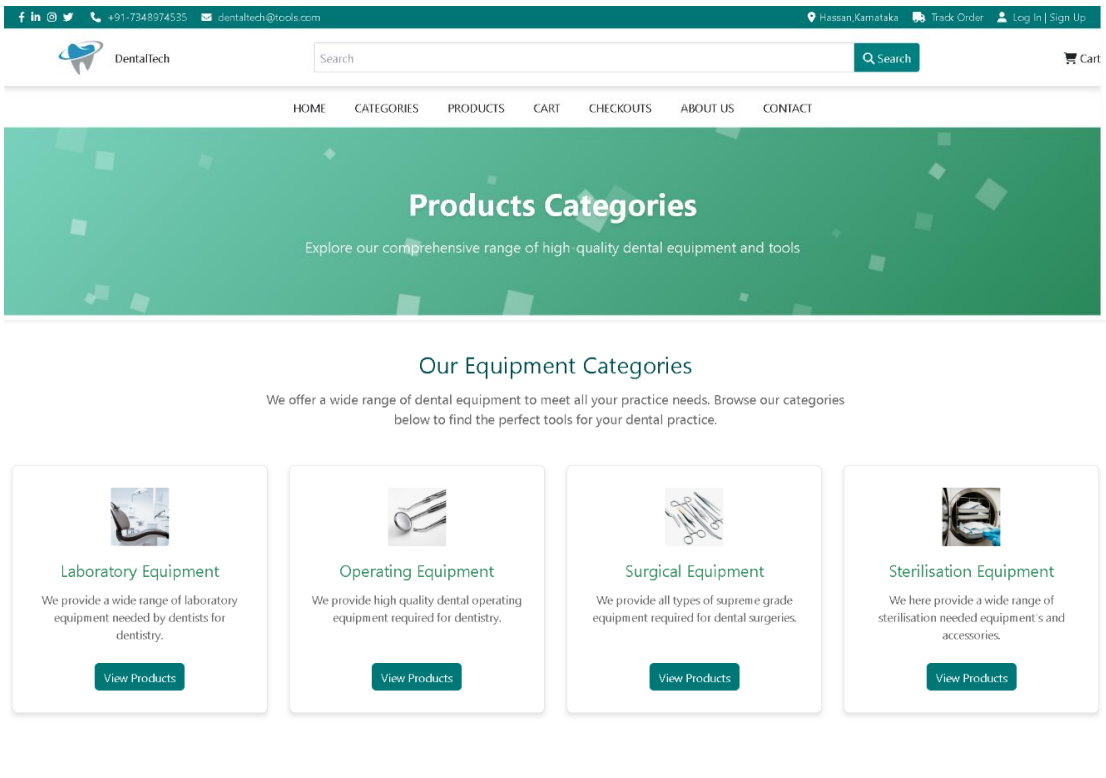

Figure 8.4: *Category Page*

8.5 PRODUCT PAGE

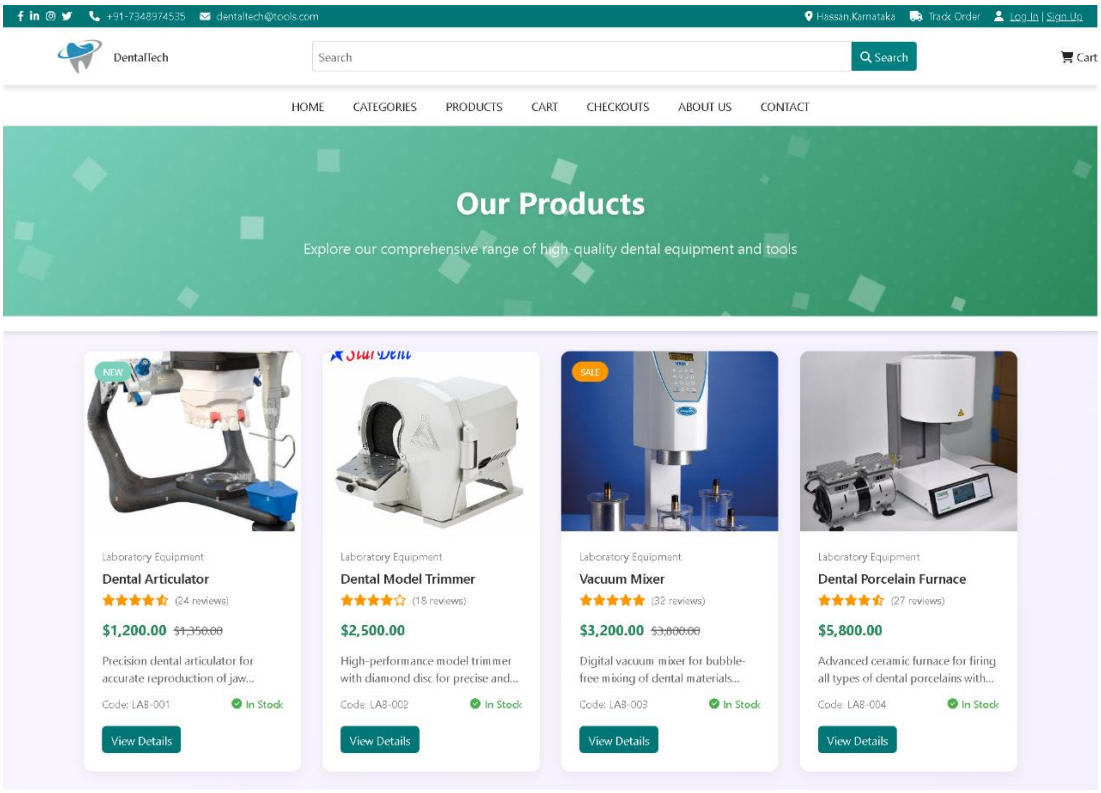

Figure 8.5 *Product Page*

8.6 CART PAGE

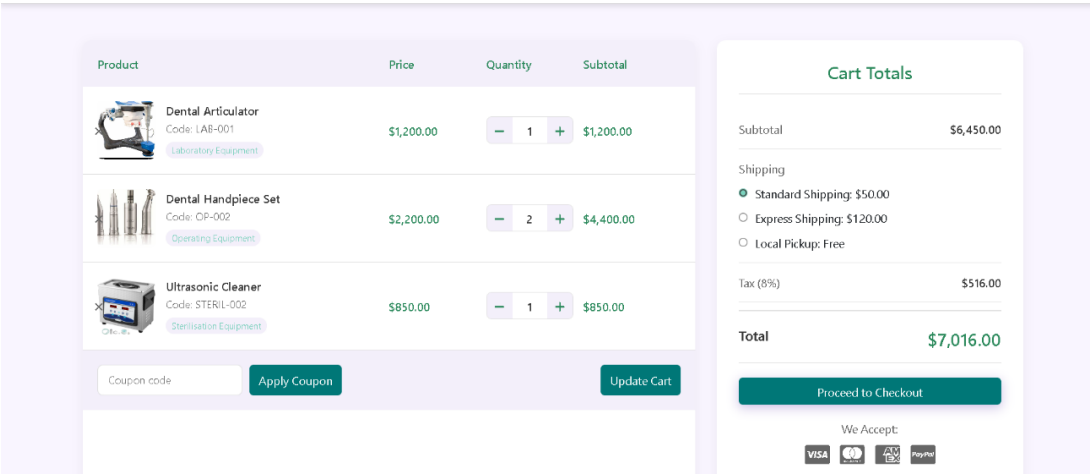

Figure 8.6: *Cart Page*

8.7 LOGIN TABLE

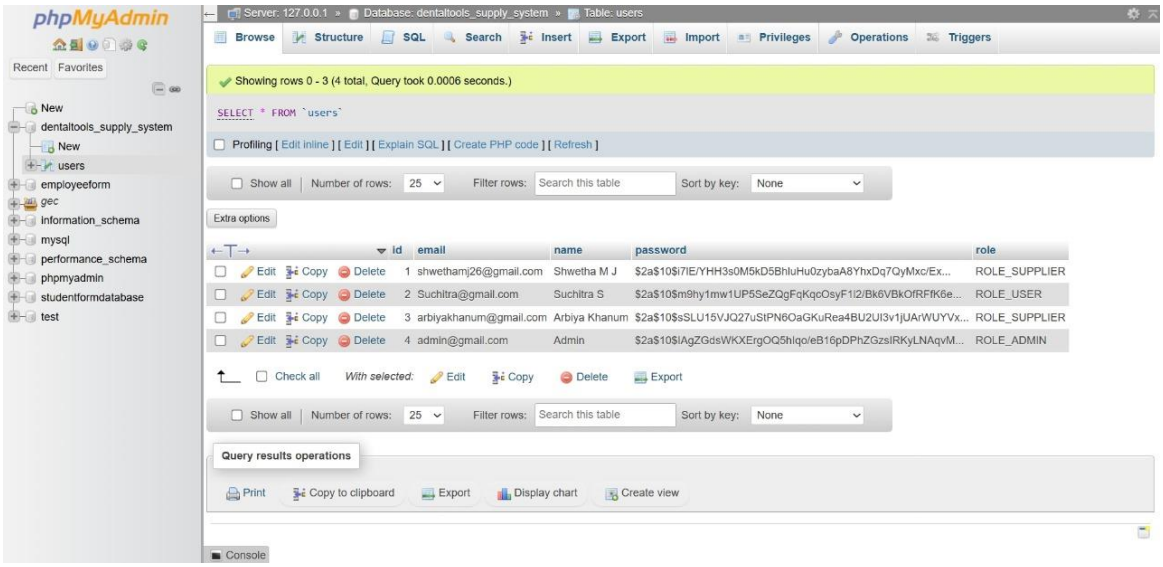

Figure 8.6: *Login Table*

8.8 SUMMARY

The Dental Tool Management System is a web-based application using Spring Boot, Thymeleaf, and MySQL to manage dental equipment, users, orders, and inventory, streamlining dental practice operations through a simple, responsive interface. It provides a centralized platform to automate and simplify dental equipment procurement and inventory management tasks. The system enhances efficiency by reducing manual record-keeping, providing real-time inventory updates, and facilitating seamless ordering processes. With role-based access control, the system ensures that users can only access features relevant to their responsibilities, maintaining data security and operational integrity. The responsive design ensures accessibly across various devices, allowing dental professionals to manage their equipment needs from anywhere.

CHAPTER 9

CONCLUSION

The Java Full Stack Web Development internship has been an immensely rewarding journey that significantly enhanced both my technical expertise and professional development. Over the course of the internship, I developed a strong foundation in building end-to-end web applications, integrating front-end technologies like HTML, CSS, JavaScript, and modern frameworks such as React (or Angular, if applicable), with robust back-end development using Java, Spring Boot, and RESTful web services. I also gained hands-on experience in database management with MySQL and used tools like Hibernate for seamless object- relational mapping. Working on real-time projects helped me understand the complete software development life cycle, including version control (Git), deployment, and debugging. The internship fostered collaboration and communication skills through teamwork, agile methodologies, and peer code reviews. Overall, this experience has prepared me to take on full stack development roles with confidence and has inspired me to continue learning and growing in the dynamic field of web development.

REFERENCES

[1] JavaTpoint. (n.d.). *Java Tutorial*. https://www.javatpoint.com/java-tutorial

[2] GeeksforGeeks. (n.d.). *Spring Boot Tutorial*. https://www.geeksforgeeks.org/spring-boot-tutorial/

[3] TutorialsPoint. (n.d.). *HTML Tutorial*. https://www.tutorialspoint.com/html/index.html

[4] TutorialsPoint. (n.d.). *CSS Tutorial*. https://www.tutorialspoint.com/css/index.html

[5] W3Schools. (n.d.). *JavaScript Tutorial*. https://www.w3schools.com/js/

[6] Baeldung. (n.d.). *Spring Boot*. https://www.baeldung.com/spring-boot

[7] Stack Overflow. (n.d.). https://stackoverflow.com